```
  }
  data_augmentation_options {
    ssd_random_crop {
    }
  }
}
```

Additional resources

Head down to the GitHub forums to properly see all the packages and extensions that are available for object detection. Take a look at the Google developers website, which is another great resource for checking coding errors and downloading updated versions of the packages.

You will find some bundle image packs on the internet that can save a lot of time to search for trained data. Websites and web scraping tools like Shutterstock, Twitter API and Google Images have many bundles in repositories for the same object type that can be used as a good training set for the programs.

Under any circumstances if the system returns an error while submitting the datasets or the packages; or the system fails to identify a package even if it is installed, then run the same commands through the main terminal as an admin and have the system dump any error files to check for the actual cause of the issue. It's recommended to keep jar code on a Netbeans IDE that will properly pinpoint where the code is actually returning a fault or simply using additional code writing tools like Notepad++ or Panda doc.

In the future segments, we'll be delving a little deeper into the code that will be used to train the modules as well as the learning algorithms for detecting objects in images.

Check up the documentation files from the Intel® TensorFlow repository to better understand the methodology behind certain coding and why particular neural networking schemes fare better than others. The repository also lets you know about the tricks behind bringing the system to a better-optimised setup for complex algorithms and products. You can learn about sidelining the project using Intel® VTune ™ amplifier and see what works best for improving machine learning methods.